

Test Automation Proof of Architecture — 4-Week Plan (with AI-Native Track)

Context: ~20 desktop/web/POS/Unix applications, ~1,500 existing manual test scenarios.
Goal: Fact-based, POC-backed recommendation of a test automation tool/toolchain, including a measured demonstration of how AI-native capabilities improve authoring speed, maintenance cost, and coverage — plus a realistic automation target and adoption roadmap.

Guiding principles

- Don't evaluate tools in the abstract — evaluate them against *this* client's estate.
 - Select POC applications that represent the hardest 20% of the estate (POS with peripherals, legacy desktop, terminal apps), not the easiest.
 - Run the POC as an **A/B comparison: traditional automation vs. AI-native automation** on the same scenarios, so the improvement is a measured fact, not a vendor claim.
 - Measure everything: script creation time, execution time, flakiness, self-healing success rate, maintenance effort after an app change, AI false-positive rate.
 - Present coverage as a fact-based number ("X% of the 1,500 scenarios are automatable, here is the breakdown"), not a promise of 100%.
-

The AI-native improvement thesis (what you will prove or disprove)

#	AI capability	Claimed improvement	How the POC measures it
1	Generative test authoring (natural language → script; LLM converts existing manual scenario steps into automated tests)	40–70% faster authoring; mass-conversion of the 1,500-scenario backlog	Time to automate same scenario: traditional vs. AI-assisted; % of AI-generated scripts runnable with < 15 min human fix-up
2	Self-healing locators / object recognition	50–80% reduction in maintenance effort after UI changes	Deliberately change UI on POC apps; measure fix time and healing success rate per tool

#	AI capability	Claimed improvement	How the POC measures it
3	Vision / model-based recognition (AI identifies controls visually)	Unlocks automation of legacy desktop, POS, and terminal apps where DOM/object APIs fail	% of "hard" scenarios automatable with vision AI that traditional object recognition cannot handle
4	Agentic / autonomous execution (computer-use agents that operate apps from intent, not scripts)	Reduced scripting for exploratory and low-frequency scenarios	Run 2–3 scenarios purely from natural-language intent; measure reliability over 10 runs
5	Visual AI validation	Catch rendering/layout defects on POS & web with near-zero assertion coding	Inject visual defects; measure detection rate and false-positive rate
6	AI test-data generation & flaky-test triage	Faster data prep; faster root-cause of failures	Time to produce valid test data sets; time to diagnose seeded failures

Honesty items to evaluate alongside the benefits (clients respect this): hallucinated or weak assertions in generated tests, non-deterministic behavior of agentic runs, data privacy when screens/data are sent to cloud AI services (crucial for POS/payment apps), token/consumption-based cost models, vendor lock-in.

Week 1 — Discovery & Estate Analysis

Day 1 — Kickoff & access

- Kickoff: confirm scope, success criteria, stakeholders, decision-makers. Position the AI-native A/B track explicitly as part of the POA.
- Request access: applications, environments, test management tool (Jira/ALM/Xray/Zephyr), CI/CD, source control.
- **AI-specific:** capture the client's AI governance posture early — is cloud AI permitted? Any data-residency or PCI/payment-data constraints (critical for POS)? On-prem/self-hosted LLM required?

Day 2 — Application inventory

- Application landscape matrix for all ~20 apps: type (web/desktop/POS/terminal), technology, version, hosting, release cadence, criticality.
- Identify cross-app end-to-end flows (these drive tool choice heavily).

- Flag apps where traditional object recognition is likely to fail — these are the prime candidates for vision-AI/agentic approaches.

Day 3 — Test scenario analysis

- Export the ~1,500 scenarios; classify by application, type, frequency, data dependency, environment dependency, manual-only constraints (hardware, printers, card readers).
- Tag each scenario: *automatable now / automatable with effort / not automatable*.
- **AI-specific:** assess the *quality of scenario documentation* — well-written steps + expected results are direct fuel for LLM-based script generation. Estimate what % of the backlog is "AI-convertible as written" vs. needs rewriting first.

Day 4 — Stakeholder interviews & constraints

- Interview QA leads, dev leads, ops: pain points, past automation attempts, team skill profile, budget appetite.
- **AI-specific:** gauge team attitude toward AI-assisted work, and probe the top maintenance pain (this is where self-healing sells itself). Capture security/procurement rules for AI services.

Day 5 — Evaluation framework & tool longlist

- Weighted criteria (example): technology coverage (20%), authoring effort incl. AI-assisted (15%), maintainability & self-healing (15%), AI capability maturity & accuracy (10%), CI/CD & reporting (10%), skill fit (10%), licensing/TCO incl. AI consumption costs (12%), vendor support & roadmap (8%).
 - Longlist candidates for a mixed estate:
 - **AI-native commercial platforms:** Tricentis Tosca (Vision AI + Copilot), Testim, mabl, Functionize, testRigor, ACCELQ, Katalon (AI features), KaneAI/LambdaTest, Autify, Eggplant (image/AI-based — strong for POS), Applitools (visual AI layer).
 - **Traditional commercial:** OpenText UFT One, Ranorex, TestComplete.
 - **Open source + AI augmentation:** Playwright/Selenium + LLM-based generation (e.g., Claude/Copilot-assisted authoring, ZeroStep-style NL steps), Healenium for self-healing, Robot Framework stack, WinAppDriver/FlaUI for desktop, SSHLibrary for Unix; computer-use agents for legacy/desktop exploration.
 - **Hybrid:** open-source web stack + AI-native/vision tool for POS & legacy desktop.
 - Get client sign-off on criteria, weights, and the A/B methodology **before** the POC.
-

Week 2 — Shortlisting & POC Setup

Day 6 — Shortlist & POC scope

- Score the longlist on paper; shortlist 2-3 stacks — ensure **at least one AI-native platform and one traditional/open-source baseline** so the A/B comparison is possible.
- Select 3-4 representative apps: modern web, legacy desktop, POS, Unix/terminal.
- Select 10-15 POC scenarios: simple flow, data-driven flow, cross-app E2E, and one "hard" case per app type. Reserve 2-3 scenarios specifically for the AI tracks (NL-authored, agentic, vision-based).

Day 7 — Environment & licenses

- Trial licenses / open-source installs; provision POC machines/VMs.
- Verify each tool can see each app type (object spy / vision recognition check) — eliminates weak candidates fast.
- **AI-specific:** confirm AI features are enabled in trials (some vendors gate them), set up any LLM API access within client's security rules, and use masked/synthetic data for anything sent to cloud AI.

Day 8 — POC design & baseline scorecard

- Define the scorecard metrics per tool per scenario: time to automate, stability over N runs, object recognition quality, debugging experience.
- **AI metrics:** authoring-time delta (AI-assisted vs. manual scripting), % of generated script usable, self-healing success rate, healing false-positive rate, agentic run consistency, visual-AI defect detection rate, AI run cost per scenario.
- Build framework skeletons per stack (page objects/modules, test data, reporting).

Day 9 — Build: Baseline (traditional) track

- Automate the POC scenarios the traditional way in the baseline stack. Log effort meticulously — this is the denominator for every AI improvement claim.

Day 10 — Build: Baseline (continued) + AI conversion pilot + checkpoint

- Finish baseline scenarios; run 10+ times for stability.
- **AI pilot:** feed 20-30 of the client's real manual scenarios into LLM-based generation; measure the "AI-convertible as written" rate — an early, powerful data point.
- Weekly checkpoint: estate analysis results, baseline effort numbers, first AI-conversion findings.

Week 3 — POC Execution & A/B Comparison

Days 11-12 — Build: AI-native platform track

- Automate the *same* scenarios in the AI-native platform using its NL/generative authoring and AI object recognition. Capture authoring time for a direct A/B against Day 9-10 baseline.
- Note where AI recognition succeeds on controls the traditional tool couldn't handle (especially POS/terminal).

Day 13 — Build: Third stack / agentic & vision experiments

- Third shortlisted stack (often the hybrid), plus the experimental scenarios: 1-2 agentic runs from pure natural-language intent, and visual-AI validation on the POS UI.
- Record reliability honestly — agentic approaches often shine on demo day and wobble on run 7; you need the run-consistency data.

Day 14 — Integration & operability

- Each stack: CI/CD integration (Jenkins/Azure DevOps/GitLab), triggered runs, published reports, results back to Jira/ALM, parallel execution.
- **AI-specific:** verify self-healing events are logged/auditable in CI, and check how AI features behave in headless/pipeline mode vs. IDE mode.

Day 15 — Maintainability & resilience A/B test (the money slide)

- Deliberately change UI elements/locators in the POC apps. Measure per stack: (a) how many tests break, (b) self-healing success rate, (c) human fix time. This traditional-vs-AI maintenance comparison is usually the most persuasive chart in the final deck.
- Run all suites 20+ times; record flakiness and any AI false positives/negatives.
- Collect licensing quotes incl. AI consumption pricing (3-year TCO view).

Week 4 — Analysis, Recommendation & Roadmap

Day 16 — Consolidate POC data

- Fill the weighted scorecard with measured data only.

- Compute per stack: avg. automation time per scenario → extrapolate effort to convert the full automatable backlog, **with and without AI assistance** (e.g., "1,280 scenarios: ~14 person-months traditional vs. ~6 with AI-assisted generation").

Day 17 — Coverage & ROI model

- Coverage statement, now in four buckets: automatable traditionally / automatable **only** via AI-vision-agentic approaches / needs app or testability changes / remains manual. The AI-only bucket is your headline: coverage the client could not reach before.
- ROI model: manual regression cost vs. automated; add AI deltas — authoring savings, maintenance savings from self-healing, minus AI licensing/consumption costs; break-even timeline per stack.

Day 18 — Recommendation & roadmap

- Final recommendation traced to criteria and POC evidence — often a hybrid: AI-native platform where it measurably wins (authoring at scale, POS/legacy recognition, self-healing) + open-source where it's cost-effective.
- Adoption roadmap: framework build-out, AI-assisted mass conversion of the scenario backlog (with human review gates), pilot apps, team training (prompting/reviewing AI-generated tests is a new skill), governance (review gates for AI-generated assertions, self-healing audit policy, data-masking policy for AI services), 6/12/18-month milestones.

Day 19 — Report & presentation prep

- POA report: estate analysis, evaluation framework, A/B POC results, AI improvement metrics, risks (incl. AI-specific), recommendation, roadmap, cost model.
- Executive deck (10-12 slides). Lead with three measured numbers: authoring-time reduction, maintenance-effort reduction, coverage unlocked by AI. Technical appendix behind it.
- Internal dry run / peer review.

Day 20 — Presentation & handover

- Present: exec summary first, evidence on demand. Demo one live A/B moment if possible (e.g., break a locator, show self-healing vs. traditional failure).
 - Hand over POC code, scorecards, raw data, AI-generated-script samples with review notes.
 - Agree next steps: procurement, pilot team, conversion project kickoff.
-

Deliverables checklist

1. Application landscape & technology matrix (with AI-suitability flags)
2. Scenario classification with automatability tagging + AI-convertibility rate
3. Weighted evaluation framework (client-approved, AI criteria included)
4. POC scripts + A/B scorecard: traditional vs. AI-native, per metric
5. Coverage feasibility statement incl. "unlocked by AI" bucket
6. 3-year TCO & ROI model incl. AI consumption costs and savings
7. Recommendation report + executive presentation
8. Adoption roadmap incl. AI governance and backlog-conversion plan

Key risks to manage

- **Access/license delays** — raise Day 1, escalate Day 3; AI features in trials sometimes need vendor enablement.
- **Cherry-picked POC scenarios** — insist on the hardest app types; AI claims must survive POS and terminal apps.
- **AI over-claiming** — validate every generated script and healing event; report false-positive rates, not just success rates.
- **Data privacy** — never send real payment/customer data to cloud AI; masked/synthetic data policy from Day 7.
- **"100% automation" expectation** — reset with data at the Week 2 checkpoint; AI raises the ceiling but doesn't remove it (hardware, exploratory judgment).
- **Tool bias** — the pre-agreed weighted scorecard and A/B methodology are your defense.